Attentional and Adversarial Feature Mimic for Efficient Object Detection

Jiasi Wang[†*] Peng Tang[†] Heng Luo[‡] Yifeng Geng[‡]
Xinyu Zhang[‡] Chang Huang[‡] Zilong Huang[†] Wenyu Liu[†] Xinggang Wang[†]
[†]Institute of AI, School of EIC, Huazhong University of Science and Technology
[‡]Horizon Robotics Inc.

Abstract

In this paper, we focus on learning efficient object detectors by knowledge (or network) distillation. More specifically, we mimic features from deeper and larger teacher networks to help train better efficient student networks. Unlike the previous method [3] that mimics features through minimizing an L2 loss between feature generated by teacher and student networks, we propose an Attentional and Adversarial Feature Mimic (AAFM) method which consists of an attentional feature mimic module and an adversarial feature mimic module, where the former module uses an attentional L2 loss which learns to pay attention to important object-related regions for feature mimic, and the latter module uses an adversarial loss which makes features generated by teacher and student networks have similar distributions. We apply our AAFM method in the two-stage Faster R-CNN detector [37]. Experiments on the PASCAL VOC 2007 and COCO datasets show that our method consistently improves the performance of detectors without feature mimic or with other feature mimic methods. In particular, our method obtains 72.1% mAP on the PASCAL VOC 2007 dataset using the ResNet-18-based detector.

1. Introduction

With the great success of Convolutional Neural Networks (CNNs) [19, 20], great leap forwards have been achieved in many computer vision applications [4, 7, 14, 28, 40, 50, 51], especially for object detection [8, 9, 13, 25, 27, 29, 35, 37, 44, 46]. One of the most important performance improvements of modern object detectors comes from the deeper and larger CNNs [2, 14, 19, 40]. However, the increase in network size results in more computation costs in the inference stage and thus is less efficient. In real applications, it is crucial for us to have detectors that are as accurate as possible and fast enough in the meantime.

There have been lots of efforts underway to speed up the process of object detection, using methods such as single shot object detectors [25, 27, 35, 36], light-weight architectures [17, 18, 23], and network pruning [15, 32]. However, these methods cannot achieve satisfactory speed-accuracy trade-offs for real applications. Recently, inspired by the knowledge distillation for image classification [16, 38], many works focus on Object Detection Network Distillation (ODND), *i.e.*, training more efficient object detection networks (*i.e.*, student networks) by mimicking the behaviors of deeper and larger networks (*i.e.*, teacher networks), and show that the accuracy of teacher networks can be recovered to some extent by student networks without any additional parameters and computation costs during inference [3, 22, 48]. In addition, network distillation appears even more promising with other technologies (*e.g.*, light-weight architectures) [31, 48]. Therefore, we focus on the ODND problem in this paper.

Inspired by the knowledge distillation method for image classification [38], the feature mimic is used in the recent ODND methods [3, 22]. More precisely, they mimic features to make features extracted from student network to be as similar as possible to features extracted from teacher network. In particular, Chen *et al.* [3] use an L2 loss to minimize the position-wise difference between feature maps generated by teacher and student networks. However, there are two potential problems in the position-wise L2 loss. First, in the case that an image only contains few objects or small objects, most of the regions on the feature map of teacher networks will response weakly, which may result in the degradation of object-related semantic information for student networks [22]. Second, this position-wise L2 loss handles each position of feature maps and each image independently, thus cannot depict the intrinsic structural correlations between the features generated by the teacher and student networks from a global view. Although it can make the two features have similar values, the two features may intrinsically have very different distributions due to their different network structures. By contrast, Li *et al.* [22] minimize the proposal-wise L2 loss between object proposal features extracted by the teacher and student

[*]The work was done during Horizon Robotics Inc. intership.

networks, which solves the first problem to some extent. However, this method handles each proposal feature and each image independently, which still suffers from the second problem. Motivated by these facts, we propose an Attentional and Adversarial Feature Mimic (AAFM) method, which consists of an attentional feature mimic module to selectively pay attention to different regions and an adversarial feature mimic module to transfer features over global distribution, to solve these two problems.

In our method, to make the student network learns from the teacher, we first train a large network as a teacher and then forward each image through both the teacher and student networks to obtain the corresponding feature maps for our feature mimic modules. More specifically, the attentional feature mimic module generates a spatial attention map to assign weights to each position for the L2 loss. Similar to the original L2 loss [3], the attentional L2 loss also handles each position independently, and thus mimic features in a local view. The difference is that the spatial attention map helps the training focus on more object-related regions and thus captures more object-related information for the student network. Inspired by the generative adversarial network [10, 45], the adversarial feature mimic module uses an adversarial loss to push the features generated by the teacher and student networks to be similar in a more global level, by fooling a discriminator network that is well-trained to differentiate features generated by different networks. Such a loss tries to make the features generated by the teacher and student networks have similar distributions, and thus can depict the intrinsic structural correlations between the two features to some extent. We combine the local attentional L2 loss and the global adversarial loss together because they are complementary to some extent.

We apply our method in the two-stage Faster R-CNN detector [37]. Apart from the losses for mimicking features described above, the classification and regression losses, trained by ground-truth annotations as in the standard Faster R-CNN, are also chosen, following [3, 22, 48]. In addition, a proposal-wise adversarial loss is also used to improve the R-CNN features. Elaborate experiments on the object detection datasets PASCAL VOC [6] and COCO [26] show that our method improves the performance of detectors without feature mimic or with other feature mimic losses [3, 22]. In particular, using a ResNet-101 based Faster R-CNN as a teacher, our method obtains 72.1% mAP for a ResNet-18-based Faster R-CNN.

In summary, we list our main contributions as follows.

- We proposed an attentional and adversarial feature mimic method for ODND. The method consists of an attentional feature mimic module which learns to pay attention to more object-related regions and an adversarial feature mimic module which makes features generated by teacher and student networks have similar distributions.
- To confirm the effectiveness of our feature mimic method, we apply the method in the two-stage Faster R-CNN detector. Our method improves the performance of detectors without feature mimic or with other feature mimic losses [3, 22] on the PASCAL VOC and COCO datasets.

2. Related Work

Object detection. Benefits from the great success of CNNs [19, 20], the performance of object detection has been significantly improved [8, 9, 13, 24, 25, 27, 35, 37, 42, 43]. One popular direction of the CNN-based object detection is the two-stage methods [8, 9, 13, 24, 37]. Such methods perform object detection by first generating region proposals and then classifying proposals. Very promising results are obtained by these methods. In this paper, we build our method on the two-stage methods, more specifically, the Faster R-CNN detector [37].

Object detection speedup. Although the two-stage methods obtain magnificent performance, they may still suffer from the inefficient inference process, especially when deeper and larger CNNs are used, which limits their use employment in real applications. Many excellent previous works focus on Object detection speedup. For example, the single shot object detectors [25, 27, 35, 36] remove the region proposal generation stage for speedup. Many works design light-weight architectures [17, 18, 23] or prune networks [15, 32] for high efficiency. Recently, inspired by the success of knowledge distillation for image classification [16], many works focus on the object detection network distillation. For example, Chen *et al*. [3] adopt an adaptation layer to adapt student features and use a L2 loss to mimic the intermediate features of teacher as well as the classification and regression results. Li *et al*. [22] also adopt the adaptation layer and conduct a proposal-based feature mimic. Wei *et al*. [48] follow the proposal-based feature mimic method [22] to help training tiny detectors. Besides, they quantize the teacher network to reduce the search scope of the student network. Mehta *et al*. [31] design a light-weight single shot network and combine it with distillation losses. These network distillation methods [22, 48, 31] show very promising speed-accuracy trade-offs. We also focus on the object detection network distillation and propose an attentional and adversarial feature mimic method.

Knowledge distillation. Knowledge distillation is to recover the performance gap between teacher and student networks. It is firstly proposed for image classification [16, 38]. For example, Hinton *et al*. [16] propose to use soft labels generated by outputs of teacher network as supervisions to train student network. Remero *et al*. [38] extend the idea by mimicking features generated by intermediate

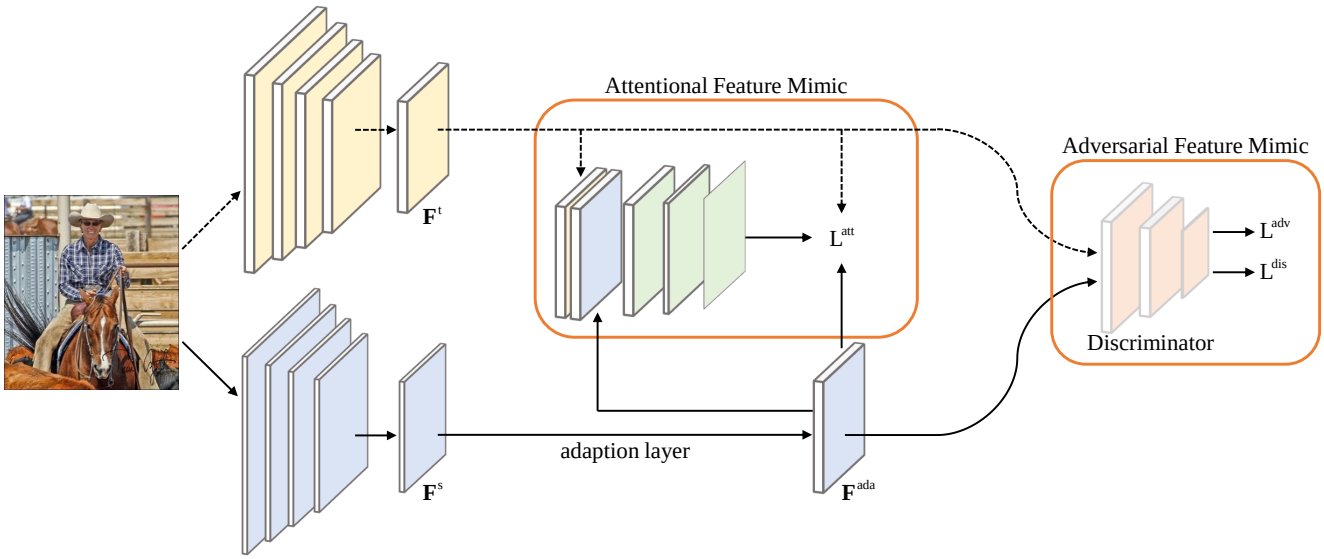

Figure 1. The overall architecture of our proposed Attentional and Adversarial Feature Mimic (AAFM) method. All arrows are utilized during the forward process of training and only the solid ones have back-propagation computations. The AAFM has two modules attentional feature mimic (see Section 3.2) and adversarial feature mimic (see Section 3.3) with losses $\mathrm{L}^{\mathrm{att}}$ (see Eq. (3)) and $\mathrm{L}^{\mathrm{adv}}$ (see Eq. (5)) respectively. Following the generative adversarial network [10], there is also a loss $\mathrm{L}^{\mathrm{dis}}$ (see Eq. (4)) to train the discriminator. During training, we alternate the optimization of the feature mimic loss and discriminator loss.

layers of teacher network to train student network. They all show that knowledge distillation improves classification results of student networks.

Others. Humans have the capability of multiple fixation points, each attended to scene content at different locations and scales [12]. Inspired by this, the attention is proposed and has been applied to many different applications, such as speech recognition [5], machine translation [1], question-answering [41], object detection [52], *etc*. Here, we use spatial attention to help student network focuses on mimicking the features of the object-related regions. Generative Adversarial Networks (GAN), proposed in [10], is a min-imax two-player game. GAN consists of a discriminator and a generator. By maxing the probability of discriminator making mistakes, it maps a signal to a vector with distribution over real data. GAN has also been used in many tasks, *e.g*., small object detection [21] and neural style [45] as well as knowledge distillation [47, 49]. Both Wang *et al*. and Xu *et al*. mimic soft targets of teacher networks for image classification network distillation. In this work, we focus on ODND, use this adversarial learning to encourage features generated by teacher and student networks having similar distributions, and thus mimic features from a global view.

3. Attentional and Adversarial Feature Mimic

In this section, we will first describe our overall feature mimic procedure (see Section 3.1), and then introduce the details of the proposed attentional feature mimic module (see Section 3.2) and adversarial feature mimic module (see Section 3.3). Here we mainly describe the training procedures since the feature mimic method is used to distill knowledge only during training.

3.1. The Overall Feature Mimic Procedure

The overall architecture of our method is shown in Fig. 1. Given an input image $\mathbf{I}$ and a well-trained teacher network, the image $\mathbf{I}$ is first fed into the teacher network and student network, which produces two feature maps $\mathbf{F}^{\mathrm{t}} \in \mathbb{R}^{C^{\mathrm{t}} \times H \times W}$ and $\mathbf{F}^{\mathrm{s}} \in \mathbb{R}^{C^{\mathrm{s}} \times H \times W}$, where $\mathbf{F}^{\mathrm{t}}, \mathbf{F}^{\mathrm{s}}, C^{\mathrm{t}}, C^{\mathrm{s}}$, H and W are feature map from the teacher network, feature map from the student network, feature dimension on each position of $\mathbf{F}^{\mathrm{t}}$, feature dimension on each position of $\mathbf{F}^{\mathrm{s}}$, height and width of the feature maps, respectively. The different C^{t} and C^{s} are caused by the different architectures of teacher and student networks. To make the feature maps produced by the two networks have the same shape, an adaption layer (a simple 1×1 convolutional layer here) with parameters $\mathbf{\Theta}^{\mathrm{ada}}$ is then adopted to adapt $\mathbf{F}^{\mathrm{s}}$ to $\mathbf{F}^{\mathrm{ada}}(\mathbf{F}^{\mathrm{s}}; \mathbf{\Theta}^{\mathrm{ada}})$, following [3, 22, 38]. We use $\mathbf{F}^{\mathrm{ada}}$ later for simplification, dropping the dependence on $\mathbf{F}^{\mathrm{s}}, \mathbf{\Theta}^{\mathrm{ada}}$. After that, $\mathbf{F}^{\mathrm{t}}$ and $\mathbf{F}^{\mathrm{ada}}$ are fed into the attentional feature mimic module and the adversarial feature mimic module for feature mimic.

For the attentional feature mimic module, we choose an attentional L2 loss $\mathrm{L}^{\mathrm{att}}\left(\mathbf{F}^{\mathrm{t}}, \mathbf{F}^{\mathrm{ada}}, \mathbf{M}^{\mathrm{att}}(\mathbf{F}^{\mathrm{t}}, \mathbf{F}^{\mathrm{ada}}; \mathbf{\Theta}^{\mathrm{att}})\right)$, defined in later Eq. (3), to measure the distance between $\mathbf{F}^{\mathrm{t}}$ and $\mathbf{F}^{\mathrm{ada}}$ from a local view, where $\mathbf{M}^{\mathrm{att}}(\mathbf{F}^{\mathrm{t}}, \mathbf{F}^{\mathrm{ada}}; \mathbf{\Theta}^{\mathrm{att}}) \in \mathbb{R}^{H \times W}$ is a spatial attention map which is obtained by an

attention module with parameters $\mathbf{\Theta}^{\text{att}}$. We will give more details about this module in Section 3.2.

For the adversarial feature mimic module, suppose we have a well-trained discriminator network with parameters $\mathbf{\Theta}^{\text{dis}}$ which is trained to differentiate $\mathbf{F}^{\text{t}}$ and $\mathbf{F}^{\text{ada}}$, we choose an adversarial loss $\text{L}^{\text{adv}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{dis}}\right)$, defined in later Eq. (5), to measure to distance between $\mathbf{F}^{\text{t}}$ and $\mathbf{F}^{\text{ada}}$ from a global view. In addition, the loss to train the discriminator network is $\text{L}^{\text{dis}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{dis}}\right)$, as defined in Eq. (4). We will give more details about this module in Section 3.3.

The local attentional L2 loss and global adversarial loss should be complementary, because they focus on different aspects for feature mimic (see the rest of this section for how they are different). Therefore, our final feature mimic loss $\text{L}^{\text{mimic}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{s}}; \mathbf{\Theta}^{\text{ada}}, \mathbf{\Theta}^{\text{att}}, \mathbf{\Theta}^{\text{dis}}\right)$ is written as Eq. (1), where $\lambda^{\text{att}}, \lambda^{\text{adv}}$ are hyper-parameters to balance the two losses.

$$\begin{aligned} &\text{L}^{\text{mimic}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{s}}; \mathbf{\Theta}^{\text{ada}}, \mathbf{\Theta}^{\text{att}}, \mathbf{\Theta}^{\text{dis}}\right) \\ &= \lambda^{\text{att}}\text{L}^{\text{att}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}, \mathbf{M}^{\text{att}}(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{att}})\right) \\ &+ \lambda^{\text{adv}}\text{L}^{\text{adv}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{dis}}\right). \end{aligned} \tag{1}$$

Following the generative adversarial network [10], we use an alternating training strategy, *i.e.*, we alternate the following two training steps: optimizing the loss $\text{L}^{\text{mimic}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{s}}; \mathbf{\Theta}^{\text{ada}}, \mathbf{\Theta}^{\text{att}}, \mathbf{\Theta}^{\text{dis}}\right)$ w.r.t. $\mathbf{F}^{\text{s}}, \mathbf{\Theta}^{\text{ada}}, \mathbf{\Theta}^{\text{att}}$ and fixing $\mathbf{F}^{\text{t}}, \mathbf{\Theta}^{\text{dis}}$; optimizing the loss $\text{L}^{\text{dis}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{dis}}\right)$ w.r.t. $\mathbf{\Theta}^{\text{dis}}$ and fixing $\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{s}}, \mathbf{\Theta}^{\text{ada}}, \mathbf{\Theta}^{\text{att}}$. Accordingly, the student network is learned from the teacher network.

3.2. Attentional Feature Mimic Module

The attentional feature mimic module mimics features produced by teacher networks in a local view. More precisely, the module wants the features in each position of $\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}$ to be same. The most straightforward way is to use the position-wise L2 loss as in Eq. (2), following [3].

$$\text{L}^{\text{L2}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}\right) = \frac{1}{H \times W}\sum_{h=1}^{H}\sum_{w=1}^{W}\|\mathbf{f}^{\text{t}}_{hw} - \mathbf{f}^{\text{ada}}_{hw}\|_2^2, \tag{2}$$

where $\mathbf{f}^{*}_{hw}$ is the feature vector with spatial position h, w of the feature map $\mathbf{F}^{*}$.

However, for object detection, the features of object regions are more important than the features of backgrounding regions for bounding box classification and regression. When the student network is very tiny [31, 22, 48], of which the capacity is limited, focusing too much on mimicking the background features will lead to poor performance. Especially, in the case that an image only contains few objects or small objects, most of the regions of the image will be background, and then $\mathbf{F}^{\text{t}}$ will have weak responses on these regions. As a result, the loss in Eq. (2) will lean to the weak background responses, which will result in the degradation of object-related semantic information for the student network. To deal with this problem, we design an attentional L2 loss as in Eq. (3), where M^{att}_{hw} is the element with spatial position h, w of $\mathbf{M}^{\text{att}}(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{att}})$ and λ^{norm} is a hyper-parameter to balance the two terms in Eq. (3).

$$\begin{aligned} &\text{L}^{\text{att}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}, \mathbf{M}^{\text{att}}(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{att}})\right) \\ &= \frac{1}{H \times W}\sum_{h=1}^{H}\sum_{w=1}^{W} M^{\text{att}}_{hw}\|\mathbf{f}^{\text{t}}_{hw} - \mathbf{f}^{\text{ada}}_{hw}\|_2^2 \\ &+ \lambda^{\text{norm}} / \sum_{h=1}^{H}\sum_{w=1}^{W}(M^{\text{att}}_{hw})^2. \end{aligned} \tag{3}$$

The explanation of the loss is as follows. Different regions should have different importances. For object detection, object regions and their surrounding context regions should be more important than other regions. Therefore, we use an attention layer which learns to pay different attentions on different regions. More specifically, the attention layer generates a spatial attention map, and accordingly different weights are assigned to different spatial positions of the position-wise L2 loss. See the first term of Eq. (3). However, directly optimizing the first term of Eq. (3) without any restriction will result in that all M^{att}_{hw} turn to be zero rapidly. To handle this problem, we add an extra penalty term to ensure not all M^{att}_{hw} to be zero, see the second term of Eq. (3).

The detailed architecture of the attention layer is described as follows. As shown in Fig. 1, to obtain the spatial attention map, we stack the two feature maps $\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}$ and pass it through three 3×3 convolutional layers with stride 1. The numbers of the output channels of the three convolutional layers are 1024, 1024, and 1, respectively. Each convolutional layer is followed by a ReLU layer [33].

3.3. Adversarial Feature Mimic Module

Although the attentional L2 loss learns some object-related semantic information for the student network, it handles each position of feature maps and each image independently, which cannot ensure the features generated by the teacher and student networks have similar distributions, and thus cannot depict the intrinsic structural correlations between features generated by the two networks. To aid this problem, the adversarial feature mimic module mimics features in a more global view.

As we described in Section 3.1, the adversarial feature mimic module requires to train a discriminator. Therefore, we first describe our discriminator here. Similar to [45], the discriminator takes a feature map $\mathbf{F}$ as input, and outputs a probability map $\mathbf{P}(\mathbf{F}; \mathbf{\Theta}^{\text{dis}}) \in \mathbb{R}^{H' \times W'}$, where H' and W' are the height and width of the map respectively. Here we use three 3×3 convolutional layers with stride 2 as the discriminator here, where the numbers of the output

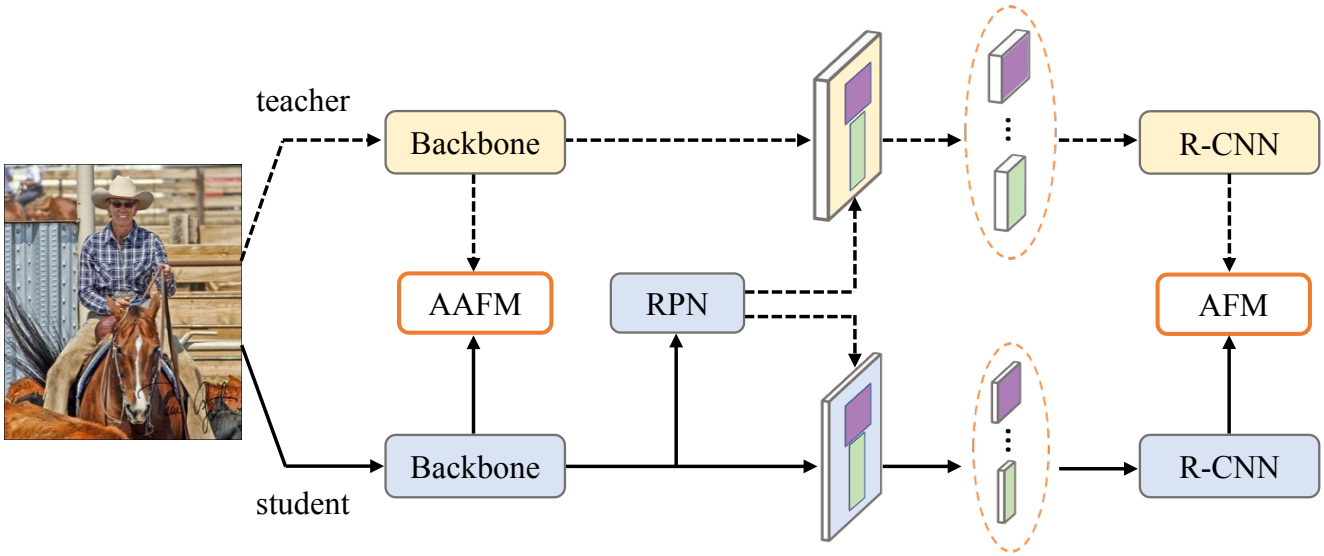

Figure 2. The overall architecture of our Faster R-CNN with attentional and adversarial feature mimic. All arrows are utilized during the forward process of training and only the solid ones have back-propagation computations. We mimic features for both of the backbone network and the R-CNN. The "AAFM" denotes that we apply the attentional and adversarial feature mimic for the backbone network. The "AFM" denotes that we apply the adversarial feature mimic for the R-CNN. The standard Faster R-CNN losses, trained by ground-truth annotations, are chosen and omitted in the figure.

channels of the three layers are 1024, 512, and 1, respectively. The first two layers as followed by a leaky ReLU layer [30] parameterized by 0.2, and the final layer is followed by a sigmoid layer to ensure the outputs within the range 0∼1. We set the kernel size and stride of the layers to be larger than 1, because we want each position of the probability map corresponds to a large region of the feature maps, and thus can capture enough structural information of the feature maps. The loss to train the discriminator is written as in Eq. (4), which is designed to differentiate against $\mathbf{F}^{\text{t}}$ and $\mathbf{F}^{\text{ada}}$, where P_{hw} is the element with spatial h, w of $\mathbf{P}(\mathbf{F}; \mathbf{\Theta}^{\text{dis}})$.

$$\begin{aligned} &\text{L}^{\text{dis}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{dis}}\right) \\ &= \begin{cases} -\sum\limits_{h=1}^{H'}\sum\limits_{w=1}^{W'} \log(P_{hw})/(H' \times W') & \text{if } \mathbf{F} = \mathbf{F}^{\text{t}}, \\ \sum\limits_{h=1}^{H'}\sum\limits_{w=1}^{W'} \log(P_{hw})/(H' \times W') & \text{else.} \end{cases} \end{aligned} \tag{4}$$

Suppose that we already have a well-trained discriminator, the adversarial loss is written as in Eq. (5), where P^{ada}_{hw} is the element with spatial h, w of $\mathbf{P}(\mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{dis}})$.

$$\text{L}^{\text{adv}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{dis}}\right) = -\frac{1}{H' \times W'}\sum_{h=1}^{H'}\sum_{w=1}^{W'} \log(P^{\text{ada}}_{hw}). \tag{5}$$

As we can see, by optimizing $\text{L}^{\text{adv}}\left(\mathbf{F}^{\text{t}}, \mathbf{F}^{\text{ada}}; \mathbf{\Theta}^{\text{dis}}\right)$, we are trying to fool the discriminator to make $\mathbf{F}^{\text{ada}}$ as similar as $\mathbf{F}^{\text{t}}$. In addition, unlike the attentional L2 loss which makes features generated by the teacher and student networks have the exact same value, the adversarial loss tries to make the two features have similar distributions over the training dataset, which depicts the intrinsic structural correlations between the two features.

4. Faster R-CNN with Attentional and Adversarial Feature Mimic

In this section, we will describe how to apply our Attentional and Adversarial Feature Mimic (AAFM) method to object detection. Here we choose the typical two-stage detector Faster R-CNN [37] which is the foundation of many recent state-of-the-art detectors [13, 24]. If not specified, the contents we described here are the training procedures.

The Faster R-CNN detector consists of three parts: a backbone network that produces convolutional feature maps; a Region Proposal Network (RPN) that generates region proposals; a Region-based CNN (R-CNN) that performs bounding box (*i.e.*, proposal) classification and regression, where the RPN and R-CNN are built upon the backbone network. Given an input image $\mathbf{I}$, the detector first feeds it into the backbone network to produce feature map $\mathbf{F}^{\text{b}}$. The RPN take $\mathbf{F}^{\text{b}}$ as input to generate region proposals $\mathcal{R}(\mathbf{F}^{\text{b}}; \mathbf{\Theta}^{\text{rpn}})$, where $\mathbf{\Theta}^{\text{rpn}}$ denotes parameters of the RPN. We use $\mathcal{R}$ later for simplification, dropping the dependence on $\mathbf{F}^{\text{b}}, \mathbf{\Theta}^{\text{rpn}}$. The R-CNN takes $\mathbf{F}^{\text{b}}$ and $\mathcal{R}$ as inputs to generate final detection results $\mathcal{D}(\mathbf{F}^{\text{b}}, \mathcal{R}; \mathbf{\Theta}^{\text{r-cnn}})$, where $\mathbf{\Theta}^{\text{r-cnn}}$ denotes parameters of the R-CNN. Then the loss to train the Faster R-CNN is written as Eq. (6), where $\mathcal{G}$ denotes the

ground truths. The loss is optimized w.r.t. $\mathbf{F}^{\text{b}}, \mathbf{\Theta}^{\text{rpn}}, \mathbf{\Theta}^{\text{r-cnn}}$ to train the detector. More details of the Faster R-CNN and its losses can be found in [37].

$$\begin{aligned} &\mathrm{L}^{\text{det}}\left(\mathbf{F}^{\text{b}}, \mathcal{G}; \mathbf{\Theta}^{\text{rpn}}, \mathbf{\Theta}^{\text{r-cnn}}\right) \\ &= \mathrm{L}^{\text{rpn}}\left(\mathbf{F}^{\text{b}}, \mathcal{G}; \mathbf{\Theta}^{\text{rpn}}\right) + \mathrm{L}^{\text{r-cnn}}\left(\mathbf{F}^{\text{b}}, \mathcal{R}, \mathcal{G}; \mathbf{\Theta}^{\text{r-cnn}}\right). \end{aligned} \tag{6}$$

We first train a large teacher detection network. For the student network, it is important to make all the three parts (*i.e.*, backbone network, RPN, and R-CNN) stronger if we want to obtain better detection performance. Since almost all Faster R-CNN based detectors [13, 24, 37] share the same and simple architecture of the RPN head, we only mimic features for the backbone network and the R-CNN. The overall architecture of our Faster R-CNN with AAFM is shown in Fig. 2.

Backbone network feature mimic. Suppose the features generated by the teacher network backbone and student network backbone are $\mathbf{F}^{\text{b_t}}$ and $\mathbf{F}^{\text{b_s}}$ respectively. It is straightforward to use the method described in Section 3 and the loss Eq. (1) because the backbones generate feature maps for whole input images. Then the loss for backbone network feature mimic is written as

$$\mathrm{L}^{\text{b_mimic}}\left(\mathbf{F}^{\text{b_t}}, \mathbf{F}^{\text{b_s}}; \mathbf{\Theta}^{\text{b_ada}}, \mathbf{\Theta}^{\text{b_att}}, \mathbf{\Theta}^{\text{b_dis}}\right), \tag{7}$$

where $\mathbf{\Theta}^{\text{b_ada}}$, $\mathbf{\Theta}^{\text{b_att}}$, and $\mathbf{\Theta}^{\text{b_dis}}$ are parameters of the backbone feature adaption layer, the backbone attention module (for attentional feature mimic), and the backbone discriminator (for adversarial feature mimic), respectively. For the adversarial feature mimic module, there is also a loss as in Eq. (4) to train the discriminator as

$$\mathrm{L}^{\text{b_dis}}\left(\mathbf{F}^{\text{b_t}}, \mathbf{F}^{\text{b_ada}}; \mathbf{\Theta}^{\text{b_dis}}\right), \tag{8}$$

where $\mathbf{F}^{\text{b_ada}}$ is the feature produced by the backbone feature adaption layer.

R-CNN feature mimic. Suppose the proposal feature generated by the teacher R-CNN and student R-CNN are $\mathbf{F}^{\text{r_t}}(\mathbf{F}^{\text{b_t}}, \mathcal{R}^{\text{s}}; \mathbf{\Theta}^{\text{r-cnn_t}})$ and $\mathbf{F}^{\text{r_s}}(\mathbf{F}^{\text{b_s}}, \mathcal{R}^{\text{s}}; \mathbf{\Theta}^{\text{r-cnn_s}})$ respectively, where $\mathcal{R}^{\text{s}}$, $\mathbf{\Theta}^{\text{r-cnn_t}}$ and $\mathbf{\Theta}^{\text{r-cnn_s}}$ are proposals generated by the student RPN, parameters of the teacher and student R-CNN, respectively. We use $\mathbf{F}^{\text{r_t}}, \mathbf{F}^{\text{r_s}}$ later for simplification, dropping the dependence on $\mathbf{F}^{\text{b_t}}$, $\mathbf{F}^{\text{b_s}}$, $\mathcal{R}^{\text{s}}$, $\mathbf{\Theta}^{\text{r-cnn_t}}$, $\mathbf{\Theta}^{\text{r-cnn_s}}$. We use the adversarial loss Eq. (5) and the position-wise L2 loss Eq. (2) (without attention) here because the R-CNN already generate proposal features which capture some object-related information. Then the loss for R-CNN feature mimic is written as

$$\begin{aligned} &\mathrm{L}^{\text{r_mimic}}\left(\mathbf{F}^{\text{r_t}}, \mathbf{F}^{\text{r_s}}; \mathbf{\Theta}^{\text{r_ada}}, \mathbf{\Theta}^{\text{r_dis}}\right) \\ &= \mathrm{L}^{\text{r_adv}}\left(\mathbf{F}^{\text{r_t}}, \mathbf{F}^{\text{r_ada}}; \mathbf{\Theta}^{\text{r_dis}}\right) + \mathrm{L}^{\text{L2}}\left(\mathbf{F}^{\text{r_t}}, \mathbf{F}^{\text{r_ada}}\right), \end{aligned} \tag{9}$$

where $\mathbf{\Theta}^{\text{r_ada}}$ and $\mathbf{\Theta}^{\text{r_dis}}$ are parameters of the R-CNN feature adaption layer and the R-CNN discriminator (for adversarial feature mimic), and $\mathbf{F}^{\text{r_ada}}$ is the feature produced by the R-CNN feature adaption layer. Similarly, there is also a loss as in Eq. (4) to train the discriminator as

$$\mathrm{L}^{\text{r_dis}}\left(\mathbf{F}^{\text{r_t}}, \mathbf{F}^{\text{r_ada}}; \mathbf{\Theta}^{\text{r_dis}}\right). \tag{10}$$

Then the combined loss to train the Faster R-CNN with AAFM will be written as in Eq. (11), where λ^{det}, $\lambda^{\text{b_mimic}}$, $\lambda^{\text{r_mimic}}$ are hyper-parameters to balance the three losses, and $\mathbf{\Theta}^{\text{rpn_s}}$ denotes parameters of the student RPN.

$$\begin{aligned} &\lambda^{\text{det}}\mathrm{L}^{\text{det}}\left(\mathbf{F}^{\text{b_s}}, \mathcal{G}; \mathbf{\Theta}^{\text{rpn_s}}, \mathbf{\Theta}^{\text{r-cnn_s}}\right) \\ &+ \lambda^{\text{b_mimic}}\mathrm{L}^{\text{b_mimic}}\left(\mathbf{F}^{\text{b_t}}, \mathbf{F}^{\text{b_s}}; \mathbf{\Theta}^{\text{b_ada}}, \mathbf{\Theta}^{\text{b_att}}, \mathbf{\Theta}^{\text{b_dis}}\right) \\ &+ \lambda^{\text{r_mimic}}\mathrm{L}^{\text{r_mimic}}\left(\mathbf{F}^{\text{r_t}}, \mathbf{F}^{\text{r_s}}; \mathbf{\Theta}^{\text{r_ada}}, \mathbf{\Theta}^{\text{r_dis}}\right). \end{aligned} \tag{11}$$

The discriminator loss is written as Eq. (12), where $\lambda^{\text{b_dis}}$ and $\lambda^{\text{r_dis}}$ are hyper-parameters to balance the two losses.

$$\begin{aligned} &\lambda^{\text{b_dis}}\mathrm{L}^{\text{b_dis}}\left(\mathbf{F}^{\text{b_t}}, \mathbf{F}^{\text{b_ada}}; \mathbf{\Theta}^{\text{b_dis}}\right) \\ &+\lambda^{\text{r_dis}}\mathrm{L}^{\text{r_dis}}\left(\mathbf{F}^{\text{r_t}}, \mathbf{F}^{\text{r_ada}}; \mathbf{\Theta}^{\text{r_dis}}\right). \end{aligned} \tag{12}$$

During training, similar to the generative adversarial network [10], we alternate the optimization of Eq. (11) (w.r.t. $\mathbf{F}^{\text{b_s}}$, $\mathbf{\Theta}^{\text{rpn_s}}$, $\mathbf{\Theta}^{\text{r-cnn_s}}$, $\mathbf{\Theta}^{\text{b_ada}}$, $\mathbf{\Theta}^{\text{b_att}}$, and $\mathbf{\Theta}^{\text{r_ada}}$) and Eq. (12) (w.r.t. $\mathbf{\Theta}^{\text{b_dis}}$ and $\mathbf{\Theta}^{\text{r_dis}}$). During inference, we simply remove all parts related to the AAFM and keep the parts of Faster R-CNN. Therefore, the AAFM does not bring any extra inference computation.

5. Experiments

In this section, we will conduct thorough experiments to analyze our proposed AAFM and its components for object detection network distillation.

5.1. Experiments Setup

Dataset and evaluation metrics. We evaluate our method on the challenging PASCAL VOC 2007 dataset [6] and COCO dataset [26]. The PASCAL VOC 2007 dataset contains 9,962 images for 20 object classes and is divided into `train`, `val`, and `test` sets. Here we choose the `train+val` (*i.e.*, `trainval`) set (5,011 images) for training and the `test` set (4,951 images) for testing. The COCO dataset contains more than 200K images for 80 object classes and is divided into `train`, `val`, and `test` sets. Here we choose the `train` set (80K images) for training and the `val` set (40K images) for testing.

For all datasets, the mean of Average Precision (mAP) is chosen as the evaluation metric. For PASCAL VOC, the standard PASCAL criterion mAP@0.5 [6], *i.e.*, IoU>0.5 between ground truth bounding boxes and predicted boxes, is used for evaluation. For COCO, the standard COCO criterion mAP@[.5, .95] [26], *i.e.*, the average of mAP for IoU$\in$[0.5 : 0.05 : 0.95], is used for evaluation.

Student	Model Info.	Teacher	mAP@0.5
VGG_M	2.0G/40.0FPS	-	60.0
		VGG16 [3]	$63.8_{\uparrow 3.8}$
		VGG16	$67.6_{\uparrow 7.6}$
VGG16	16G/14.3FPS	-	70.0
ResNet-18	1.8G/52.6FPS	-	65.4
		VGG16	$69.2_{\uparrow 3.8}$
		ResNet-50	$71.8_{\uparrow 6.4}$
		ResNet-101	$72.1_{\uparrow 6.7}$
ResNet-50	3.8G/21.7FPS	-	73.1
		ResNet-101	$74.6_{\uparrow 1.5}$
ResNet-101	7.6G/8.1FPS	-	74.9

Table 1. Our experimental results of different student/teacher networks on the PASCAL VOC 2007 `test` set. The "Model Info." denotes the FLOPs/inference speed of different networks. The "-" denotes that there is no network distillation and thus serves as baselines.

Student	Model Info.	Teacher	mAP@[.5, .95]
ResNet-50	3.8G/21.7FPS	-	32.5
		ResNet-101	$33.5_{\uparrow 1.0}$
ResNet-101	7.6G/8.1FPS	-	34.3

Table 2. Our experimental results of different student/teacher networks on the COCO `val` set. The "Model Info." denotes the FLOPs/inference speed of different networks. The "-" denotes that there is no network distillation and thus serves as baselines.

Figure 3. Qualitative results of the attention maps generated by the attentional feature mimic module. Color changing from blue to red indicates attention from less to more.

Implementation details. In our experiments, we choose several popular CNN architecture as our teacher/student networks, including VGG_M [2], VGG16 [40], ResNet-18, ResNet-50, and ResNet-101 [14] which are pre-trained on the ImageNet [39] dataset. Following the standard settings of the Faster R-CNN [14, 37], for the VGG_M and VGG16, the convolutional (conv) layers are chosen as the backbone network, two fully connected (fc) layers along with two fc layers for bounding box classification and regression are chosen as the R-CNN; for the ResNet-based architectures, the $1^{\text{st}} \sim 4^{\text{th}}$ conv blocks are chosen as the backbone network, the 5th conv block along with two fc layers for bounding box classification and regression are chosen as the R-CNN; for the RPN, one 3×3 conv layer with stride 1 and output channel 512 along with two 1×1 conv layers for proposal generation are chosen.

For backbone network feature mimic, the architectures for the feature adaption, attention, and discriminator are the conv layers described in Section 3, and the feature mimic is performed on the final outputs of the backbone network. For the R-CNN feature mimic, we use one fc layer for feature adaption and three fc layers as the discriminator, because the features generated by R-CNN are feature vectors instead of feature maps and thus we cannot use conv layers upon them.

The RoIAlign [13] is chosen to crop feature maps for each proposal. We resize the shortest side of images to 600 during training and testing, following the standard Faster R-CNN setting [37]. We train our network using stochastic gradient descent with momentum 0.9 and weight decay 0.0005. The mini-batch size is set to 4 during training. The learning rate is set to 0.01 for the first 13 and 6 training epochs and is decreased to 0.001 for the next 5 and 3 training epochs on the PASCAL VOC 2007 and COCO datasets respectively. The hyper-parameters λ^{att}, λ^{adv}, λ^{norm}, λ^{det}, $\lambda^{\text{b_mimic}}$, $\lambda^{\text{r_mimic}}$, $\lambda^{\text{b_dis}}$, and $\lambda^{\text{r_dis}}$ are set to 5, 0.00005, 10, 1, 1, 1, 1, and 1, respectively. When the feature maps generated by the teacher and student networks have a large gap in spatial shape caused by pooling or stride, we follow the padding trick [11] to match them. Otherwise, for the only several pixel distances caused by kernel size, we simply crop the larger feature map to match the shape of the smaller feature map.

Our experiments are implemented based on the PyTorch [34] deep learning framework. All of our experiments are running on the NVIDIA GTX TitanX GPU.

5.2. Overall Results

We report our results on the PASCAL VOC 2007 dataset in Table 1. As we can see, without network distillation, it is hard to achieve a good speed-accuracy trade-off. More precisely, the efficient networks cannot obtain satisfactory accuracy and the larger and deeper networks cannot achieve satisfactory speed, *e.g.*, speed 52.6FPS *vs.* 8.1FPS and accuracy 65.4% mAP@0.5 *vs.* 74.9% mAP@0.5 for ResNet-18 *vs.* ResNet-101. After our designed network distillation method, the ResNet-18 achieves 72.1% mAP@0.5 which is much better than the result without distillation and even close to the 73.1% mAP@0.5 result of ResNet-50. In particular, using VGG_M as the student network and VGG16 as the teacher network, our method obtains much better performance than the previous method [3] which uses the position-wise L2 loss in Eq. (2) (67.6% mAP@0.5 *vs.* 63.8% mAP@0.5).

L2	L2_proposal	Att	Adv	R-CNN_mimic	mAP@0.5
					65.4
✓					$69.3_{\uparrow 3.9}$
	✓				$69.7_{\uparrow 4.3}$
		✓			$70.0_{\uparrow 4.6}$
		✓	✓		$70.9_{\uparrow 5.5}$
		✓	✓	✓	$\mathbf{71.8}_{\uparrow \mathbf{6.4}}$

Table 3. Experimental results of different feature mimic designs on the PASCAL VOC 2007 `test` set. Here we use ResNet-50 as the teacher network and ResNet-18 as the student network. "L2", "L2_proposal", "Att", "Adv", and "R-CNN_mimic" denote the position-wise L2 loss used in [3], the proposal-wise L2 loss used in [22], the attentional L2 loss described in Eq. (3) for backbone network feature mimic, the adversarial loss described in Eq. (5) for backbone network feature mimic, and the loss described in Eq. (9) for R-CNN feature mimic, respectively.

We also report our results on the COCO dataset in Table 2. As we can see, we have a similar observation that our method improves the detection performance of student networks.

We show some qualitative results of the attention maps generated by the attentional feature mimic module in Fig. 3. As we can see, object-related regions capture more attentions than other regions, which confirms our theory in Section 3.2 that object-related regions are more important than other regions for object detection.

5.3. Ablation Study

In this part, we conduct elaborate experiments to analyze the influence of different feature mimic designs. Without loss of generality, we only use the ResNet-50 as the teacher network and ResNet-18 as the student network, and only perform experiments on the PASCAL VOC 2007 dataset.

Attentional feature mimic module. We first discuss the influence of the attentional feature mimic module for the backbone network. As shown in Table 3, compared with the baseline without any feature mimic, only adding an attention feature mimic module obtains much better performance (70.0% mAP@0.5 *vs*. 65.4% mAP@0.5), which confirms the effectiveness of the proposed method. In particular, compared with the position-wise L2 loss [3], our method obtains better detection performance. The better performance comes from that our method learns to pay attention to the important object-related regions which are more important than other regions for object detection, and thus captures more object-related information. In addition, compared with the proposal-wise L2 loss [22] which already focuses on some object-related regions, our method also obtains better detection performance. This is because some proposals also contain many background regions which are of less importance for object detection.

Adversarial feature mimic module. We then combine the attentional feature mimic module and the adversarial feature mimic module for the backbone network. From Table 3, we can observe that the performance is improved further. This is because the adversarial loss in the module learns to depict the intrinsic structural correlations between features generated by the teacher and student networks, which mimic features from another view.

R-CNN feature mimic. We finally add the R-CNN feature mimic module, by adding the R-CNN feature mimic loss Eq. (9). As we can see from Table 3, adding such loss obtains the best performance, which confirms our theory in Section 4 that it is important to make all parts of the Faster R-CNN detector stronger.

According to the experiments, we observe that all components of our method are important to obtain satisfactory results. In addition, using all our components outperforms the previous methods [3, 22] by a large margin. Therefore, we use all these components in our method.

5.4. Multi-stage Backbone Feature Mimic

The backbone network has multiple convolutional stages. Therefore, in this part, we study whether we can apply our feature mimic method on all these stages to achieve further performance improvement.

On the PASCAL VOC 2007 dataset, using ResNet-50 as the teacher network and ResNet-18 as the student network, multi-stage backbone feature mimic obtains 72.3% mAP@0.5 which is a little better than the 71.8% mAP@0.5 of only using the last stage. This is because in our detector, the RPN and R-CNN only takes features from the last stage, and thus only ensuring that the last stage produces promising features is enough. But for the detectors performing RPN and R-CNN on multi-stage features [13, 24], we believe the multi-stage feature mimic can obtain more promising results, which we would like to explore in the future.

6. Conclusion

In this paper, we focus on network distillation by mimicking features from teacher networks to train efficient object detection student networks. To this end, we propose an attentional and adversarial feature mimic method which consists of an attentional feature mimic module and an adversarial feature mimic module. The first module learns to pay attention to important object-related regions during feature mimic. The second module makes teacher network features and student network features have similar distributions. Our method is applied in the two-stage Faster R-CNN detector. Experimental results show that our method outperforms both of the detectors without feature mimic and with other feature mimic methods. As future works, we believe that our method can be combined with other object detection speedup technologies such as single shot object detectors, and can be applied to other tasks such as image classification and semantic segmentation for speedup, because our method mainly operates on convolutional features which are almost involved in all CNN-based methods.

References

[1] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014. 3

[2] K. Chatfield, K. Simonyan, A. Vedaldi, and A. Zisserman. Return of the devil in the details: Delving deep into convolutional nets. In *BMVC*, 2014. 1, 7

[3] G. Chen, W. Choi, X. Yu, T. Han, and M. Chandraker. Learning efficient object detection models with knowledge distillation. In *NIPS*, pages 742–751, 2017. 1, 2, 3, 4, 7, 8

[4] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille. Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. *TPAMI*, 40(4):834–848, 2018. 1

[5] J. K. Chorowski, D. Bahdanau, D. Serdyuk, K. Cho, and Y. Bengio. Attention-based models for speech recognition. In *NIPS*, pages 577–585, 2015. 3

[6] M. Everingham, S. A. Eslami, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The pascal visual object classes challenge: A retrospective. *IJCV*, 111(1):98–136, 2015. 2, 6

[7] Y. Gao, J. Ma, M. Zhao, W. Liu, and A. L. Yuille. Nddr-cnn: Layerwise feature fusing in multi-task cnns by neural discriminative dimensionality reduction. In *CVPR*, 2019. 1

[8] R. Girshick. Fast R-CNN. In *ICCV*, pages 1440–1448, 2015. 1, 2

[9] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, pages 580–587, 2014. 1, 2

[10] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *NIPS*, pages 2672–2680, 2014. 2, 3, 4, 6

[11] S. Gupta, J. Hoffman, and J. Malik. Cross modal distillation for supervision transfer. In *CVPR*, pages 2827–2836, 2016. 7

[12] K. Hara, M.-Y. Liu, O. Tuzel, and A.-m. Farahmand. Attentional network for visual object detection. *arXiv preprint arXiv:1702.01478*, 2017. 3

[13] K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask R-CNN. In *ICCV*, pages 2980–2988, 2017. 1, 2, 5, 6, 7, 8

[14] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016. 1, 7

[15] Y. He, X. Zhang, and J. Sun. Channel pruning for accelerating very deep neural networks. In *ICCV*, pages 1398–1406, 2017. 1, 2

[16] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. 1, 2

[17] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017. 1, 2

[18] K.-H. Kim, S. Hong, B. Roh, Y. Cheon, and M. Park. Pvanet: deep but lightweight neural networks for real-time object detection. *arXiv preprint arXiv:1608.08021*, 2016. 1, 2

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *NIPS*, pages 1097–1105, 2012. 1, 2

[20] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 1, 2

[21] J. Li, X. Liang, Y. Wei, T. Xu, J. Feng, and S. Yan. Perceptual generative adversarial networks for small object detection. In *CVPR*, pages 1951–1959, 2017. 3

[22] Q. Li, S. Jin, and J. Yan. Mimicking very efficient network for object detection. In *CVPR*, pages 7341–7349, 2017. 1, 2, 3, 4, 8

[23] Z. Li, C. Peng, G. Yu, X. Zhang, Y. Deng, and J. Sun. Light-Head R-CNN: In defense of two-stage object detector. *arXiv preprint arXiv:1711.07264*, 2017. 1, 2

[24] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *CVPR*, pages 936–944, 2017. 2, 5, 6, 8

[25] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. *TPAMI*, 2018. 1, 2

[26] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft coco: Common objects in context. In *ECCV*, pages 740–755, 2014. 2, 6

[27] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. SSD: Single shot multibox detector. In *ECCV*, pages 21–37, 2016. 1, 2

[28] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, pages 3431–3440, 2015. 1

[29] H. Luo, W. Xie, X. Wang, and W. Zeng. Detect or track: Towards cost-effective video object detection/tracking. In *AAAI*, 2019. 1

[30] A. L. Maas, A. Y. Hannun, and A. Y. Ng. Rectifier nonlinearities improve neural network acoustic models. In *ICML Workshop on Deep Learning for Audio, Speech and Language Processing*, 2013. 5

[31] R. Mehta and C. Ozturk. Object detection at 200 frames per second. *arXiv preprint arXiv:1805.06361*, 2018. 1, 2, 4

[32] D. Mittal, S. Bhardwaj, M. M. Khapra, and B. Ravindran. Recovering from random pruning: On the plasticity of deep convolutional neural networks. In *WACV*, pages 848–857, 2018. 1, 2

[33] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *ICML*, pages 807–814, 2010. 4

[34] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. Automatic differentiation in pytorch. 2017. 7

[35] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, pages 779–788, 2016. 1, 2

[36] J. Redmon and A. Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018. 1, 2

[37] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. *TPAMI*, 39(6):1137–1149, 2017. 1, 2, 5, 6, 7

[38] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio. Fitnets: Hints for thin deep nets. *arXiv preprint arXiv:1412.6550*, 2014. 1, 2, 3

[39] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. ImageNet large scale visual recognition challenge. *IJCV*, 115(3):211–252, 2015. 7

[40] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015. 1, 7

[41] S. Sukhbaatar, J. Weston, R. Fergus, et al. End-to-end memory networks. In *NIPS*, pages 2440–2448, 2015. 3

[42] P. Tang, C. Wang, X. Wang, W. Liu, W. Zeng, and J. Wang. Object detection in videos by short and long range object linking. *arXiv preprint arXiv:1801.09823*, 2018. 2

[43] P. Tang, X. Wang, S. Bai, W. Shen, X. Bai, W. Liu, and A. L. Yuille. Pcl: Proposal cluster learning for weakly supervised object detection. *TPAMI*, 2018. 2

[44] P. Tang, X. Wang, A. Wang, Y. Yan, W. Liu, J. Huang, and A. Yuille. Weakly supervised region proposal network and object detection. In *ECCV*, pages 352–368, 2018. 1

[45] Y.-H. Tsai, W.-C. Hung, S. Schulter, K. Sohn, M.-H. Yang, and M. Chandraker. Learning to adapt structured output space for semantic segmentation. *arXiv preprint arXiv:1802.10349*, 2018. 2, 3, 4

[46] X. Wang, K. Chen, Z. Huang, C. Yao, and W. Liu. Point linking network for object detection. *arXiv preprint arXiv:1706.03646*, 2017. 1

[47] Y. Wang, C. Xu, C. Xu, and D. Tao. Adversarial learning of portable student networks. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018. 3

[48] Y. Wei, X. Pan, H. Qin, and J. Yan. Quantization mimic: Towards very tiny cnn for object detection. In *ECCV*, pages 274–290, 2018. 1, 2, 4

[49] Z. Xu, Y.-C. Hsu, and J. Huang. Training student networks for acceleration with conditional adversarial networks. *BMVC. British Machine Vision Association*, 2018. 3

[50] K. Zhang, Y. Huang, Y. Du, and L. Wang. Facial expression recognition based on deep evolutional spatial-temporal networks. *TIP*, 26(9):4193–4203, 2017. 1

[51] K. Zhang, W. Luo, Y. Zhong, L. Ma, W. Liu, and H. Li. Adversarial spatio-temporal learning for video deblurring. *TIP*, 28(1):291–301, 2019. 1

[52] Y. Zhu, C. Zhao, H. Guo, J. Wang, X. Zhao, and H. Lu. Attention couplenet: Fully convolutional attention coupling network for object detection. *TIP*, 28(1):113–126, 2019. 3